# Geração de Perguntas com Tópico Controlado

*Relatório parcial — reprodução do baseline do artigo T-CQG · Tiago Lima Fraga (NUSP 13859638) · Inteligência Computacional · 30/09/2026*

*Artigo-base: LI, Z.; CUKUROVA, M.; BULATHWELA, S. A novel approach to scalable and automatic topic-controlled question generation in education. LAK 2025, Dublin: ACM, 2025.*

## 1. Objetivo

O artigo propõe o T-CQG (Topic-Controlled Question Generator), um modelo que recebe um contexto e um tópico-alvo e gera uma pergunta coerente e focada naquele tópico, com o objetivo de reduzir o trabalho manual de professores na criação de perguntas para materiais didáticos. O diferencial em relação a trabalhos anteriores é controlar o sub-tópico específico do contexto, não apenas o tema geral, evitando perguntas genéricas. O modelo é leve (T5-small) e não depende de LLMs proprietários via API, reduzindo custo e preservando privacidade dos dados educacionais.

Este relatório documenta a etapa preliminar do projeto: reprodução do modelo baseline (geração de perguntas sem condicionamento por tópico), que serve de ponto de comparação para a etapa seguinte (TopicQG, condicionado por tópico), ainda não implementada.

## 2. Técnicas de Inteligência Computacional aplicadas

**Fine-tuning / Transfer Learning** — Ajuste do T5-small pré-treinado (~60M parâmetros) sobre o par contexto→pergunta.

**Geração condicionada (seq2seq)** — Arquitetura encoder-decoder: entrada é o contexto (formato answer-agnostic, sem tópico nesta etapa), saída é a pergunta gerada autoregressivamente.

**Ajuste de hiperparâmetros sob restrição de recursos** — Otimizador, learning rate e batch efetivo do artigo mantidos; nº de épocas e tamanho do subconjunto de treino adaptados ao orçamento de GPU disponível (ver seção 5).

## 3. Datasets

O baseline foi treinado sobre o SQuAD 1.1, dataset padrão de QA extrativo com 87.599 perguntas de treino, cujas respostas são spans curtos (~3 palavras) extraídos de artigos da Wikipédia. Essa origem extrativa tende a produzir perguntas fatuais e fechadas (que, quando, quem).

Como parte da exploração do projeto, identificamos e analisamos o LearningQ (Chen et al., ICWSM 2018), um dataset de 188.660 perguntas reais feitas por alunos do Khan Academy, buscando entendimento em vez de um fato pontual. Medimos a fração de perguntas contendo “why”/“how” em qualquer posição como um proxy quantitativo de abertura: 12,3% no SQuAD contra 37,5% no LearningQ — cerca de 3× mais perguntas abertas. O LearningQ é candidato a fonte de dados para a etapa de tópico (TopicQG), por já vir de um domínio educacional e produzir perguntas mais alinhadas ao objetivo do artigo.

## 4. Divisão dos dados

O split oficial treino/validação do SQuAD já separa artigos completos (442 vs. 48 artigos disjuntos), e por isso a validação oficial foi reservada integralmente como conjunto de teste, nunca usada para decisões durante o desenvolvimento. Internamente, o conjunto de treino foi ainda dividido em treino/dev para early stopping. Inicialmente essa divisão interna foi feita por linha (por pergunta), o que

permite que o mesmo parágrafo apareça em treino e dev com perguntas diferentes — uma fuga de informação sutil, já que o modelo pode ter visto aquele texto exato durante o treino. Essa divisão foi corrigida para ser feita por artigo, eliminando essa fuga.

| Conjunto | Exemplos | Artigos | Uso |
|---|---|---|---|
| Treino | 83.575 | 420 | Atualização de gradiente |
| Dev | 4.024 | 22 | Early stopping |
| Teste | 10.570 | 48 | Validação oficial do SQuAD, nunca usada em decisões |

## 5. Arquitetura do modelo e pipeline de treinamento

T5-small: encoder-decoder Transformer com ~60M parâmetros, 6 camadas no encoder e 6 no decoder, embeddings compartilhados entre encoder, decoder e camada de saída. O encoder aplica self-attention + feed-forward sobre o texto de entrada ("generate question: " + contexto); o decoder aplica self-attention + cross-attention (sobre a saída do encoder) + feed-forward para gerar a pergunta token a token, de forma autoregressiva.

Hiperparâmetros mantidos do artigo: otimizador AdamW, learning rate 1e-3, epsilon 1e-8, batch efetivo 64 (via acúmulo de gradiente, necessário pela VRAM limitada da GPU local), sequência máxima de 512 tokens no encoder e 128 no decoder.

**Desvio documentado, não erro de reprodução:** o artigo treina com os 87.599 exemplos oficiais por até 50 épocas com early stopping. Na GPU local disponível (4GB de VRAM), essa configuração foi benchmarcada em ~150 horas para completar. Por restrição de tempo, a etapa preliminar reportada na seção 6 usa um subconjunto de 8.000 exemplos por até 5 épocas (parando quando a perda de validação para de melhorar). Um treinamento com o dataset oficial completo foi iniciado e seguia em andamento no momento da escrita deste relatório.

## 6. Resultados preliminares

O checkpoint preliminar (8.000 exemplos, melhor época por perda de validação) foi avaliado sobre o conjunto de teste completo (10.570 exemplos da validação oficial do SQuAD, nunca visto durante o desenvolvimento):

| BLEU | ROUGE-1 | ROUGE-2 | ROUGE-L | METEOR | F1 |
|---|---|---|---|---|---|
| **6,76** | **26,8%** | **9,5%** | **24,5%** | **25,4%** | **23,5%** |

Esses números são naturalmente mais baixos do que os relatados no artigo, dado o subconjunto e número de épocas reduzidos — o objetivo desta etapa é validar que o pipeline (dados → tokenização → treinamento → avaliação) está correto, não reproduzir os valores finais do artigo. Um exemplo qualitativo ilustra que o modelo já gera perguntas gramaticais e sobre o tópico correto, mesmo quando divergem lexicalmente da referência:

**Referência:** *"Which artist who had a major influence on the Gothic Revival is represented in the V&A's British galleries?"*
**Gerada:** *"Who is a major influence on the Gothic Revival?"*

No treinamento completo (dataset oficial, em andamento), a perda de validação já caiu de 2,005 (época 1) para 1,935 (época 2), indicando convergência saudável.

## 7. Próximos passos

1. Concluir o treinamento completo (87.599 exemplos, dataset oficial do SQuAD) e reavaliar o baseline final.

2. Implementar o TopicQG: condicionar a geração por tópico via aprendizado contrastivo (equivalente ao MixSQuAD do artigo), possivelmente usando o LearningQ como fonte adicional de dados.

3. Avaliação final comparando baseline vs. TopicQG, e confrontando com os números reportados no artigo.

4. Extensão opcional: comparar com LLMs proprietárias (Claude, GPT, Grok) como baseline adicional.